

HANSEL: Extracting Breadcrumbs from Web Agent Trajectories for Interactive Verification

Yujin Zhang

yujinz16@uci.edu

University of California, Irvine
Irvine, California, USA

Daye Nam

daye.nam@uci.edu

University of California, Irvine
Irvine, California, USA

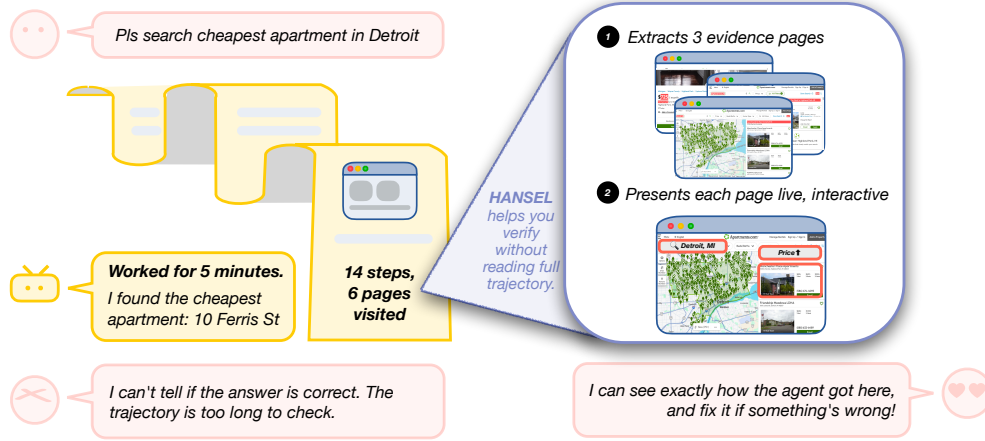


Figure 1: Verifying web agents often involves inspecting long and complex trajectories (e.g., 14 steps, 6 pages visited). HANSEL takes a web agent trajectory as input, extracts the important visited pages, and presents them as interactive pages with relevant page state preserved (e.g., applied filters, search queries, scroll positions). On each evidence page, the evidence relevant to the agent’s output is highlighted for users to verify.

Abstract

AI web agents can perform complex, multi-step tasks such as searching for products, comparing options, and making purchases on behalf of users. However, verifying the correctness of an agent’s output remains difficult. Existing transparency mechanisms — full trajectory logs, source links, screenshots, and LLM-generated summaries — treat verification as a passive reading task, leaving users to sift through overwhelming logs or trust potentially unfaithful explanations. We present HANSEL (Highlighting Agent Navigation Steps as Evidence Links), a system that extracts interactive, verifiable evidence from web-agent trajectories. Given an agent trajectory, HANSEL extracts evidence pages and snippets and presents them as navigable, interactive views with relevant page state preserved (e.g., applied filters, search queries, and scroll positions), enabling users to verify how the agent arrived at its answer. When the agent’s answer cannot be traced to any visited page, HANSEL explicitly flags this gap. A technical evaluation on 45 tasks from AssistantBench and Online-Mind2Web shows that HANSEL achieves 83.7% precision and 88.8% recall in identifying evidence pages, while reducing trajectory volume by 61.6%. In a controlled user study with 14 participants, HANSEL significantly reduced task completion time and perceived effort compared to a standard agent interface,

while participants rated it significantly higher on usability, verification ease, and error identification. Our results demonstrate that reframing verification as an interactive activity, rather than passive consumption of agent explanations, leads to more efficient human oversight of AI agents. Our supplementary materials are available at https://github.com/cloudsreal/hansel_study.git.

1 Introduction

AI agents are capable of performing multi-step, complex tasks on our behalf: agentic systems can now search for products, compare options across multiple platforms, and even make purchases or send messages on behalf of users [10, 28, 55]. As these agents take on tasks with non-trivial real consequences, a critical question emerges: how can users verify that the agent’s answer is actually correct [1, 4]?

Consider the following query posed to a web agent: “Search for the cheapest apartment in Detroit for a student on Apartments.com.” The agent must navigate Apartments.com, search for “Detroit, MI,” apply a student filter, sort prices from low to high, and return the result. Many web agents can correctly find answers to queries like this. However, when the agent returns “Manchester Place Apartments,” how likely is a user to contact the apartment manager for viewing with full trust, without verifying whether the agent missed cheaper options within the area or made an error? If the tasks had

higher stakes, for example, making a purchase or sending an email to customers, the desire to understand what agents did would only increase.

Recent systems support user monitoring and intervention during agent execution through mechanisms such as co-planning and action confirmation [21, 29]. However, requiring users to remain attentive throughout execution conflicts with the common usage scenario and motivations of agents, as users delegate tasks not because they lack the ability to perform them, but to offload tedious, repetitive, or time-consuming work [26, 32]. Rather than requiring continuous attention, systems should help users verify after execution that the agent completed the task correctly [1, 24]. Unfortunately, existing agents provide little support for post-hoc verification [14, 21]. To verify an agent’s answer, a user needs to know how the agent reached it and judge whether that process and its content are reliable and sound, yet existing transparency mechanisms fall short on these requirements. Execution logs record every page visited, click, search query, and reasoning step, which in principle allows users to trace how the agent worked, but in practice produces an overwhelming log that few users will read, let alone critically audit. Source links, provided by systems such as Gemini [16], point users to the pages the agent visited, but following a link loads the page in its current default state, losing the context needed for verification, and the links are presented as a flat collection that does not convey the agent’s reasoning process. Screenshots used by OpenAI’s Operator Search [33] capture what the agent saw on each page, but they are static and users cannot interact with them to check. Some agents also describe their process through natural-language reasoning summaries [31], but such summaries can be incomplete or hallucinated, and they can rationalize an incorrect answer as fluently as a correct one. Across these approaches, verification remains a passive task [13, 41]. Rather than presenting secondary traces of the agent’s work, a verification interface should surface task-relevant webpages directly: the actual pages the agent acted on, preserved in the state the agent left them, where users can inspect and operate on the evidence directly.

We present **HANSEL** (Highlighting Agent Navigation Steps as Evidence Links), a system that extracts and surfaces *interactive, verifiable evidence* from a Web agent’s action trajectory. HANSEL allows users to efficiently verify the agent’s action post-hoc, without active monitoring or intensive analysis. Given a trajectory, HANSEL identifies the subset of visited pages that directly support the agent’s final answer, and presents them as navigable interactive views with relevant state preserved (e.g., sorting). Figure 2 shows these core evidence pages and the support HANSEL provides. HANSEL makes unsupported reasoning visible through the evidence pages when the agent returns a wrong answer, even when it seems plausible. HANSEL also supports *recovery from partial agent failures*. When an agent fails mid-task, for example, after loading the listing but failing to sort results by price from low to high, the evidence page HANSEL has already extracted allows the user to resume from the last successful step rather than restarting the task from scratch. Users can open the listing results the agent found, apply the sorting, and manually check the cheapest apartment themselves.

The design of HANSEL is guided by three goals. First, HANSEL *reduces evidence to what matters*: rather than presenting the execution logs, which are too overwhelming, or a single visited page,

which is too narrow, HANSEL surfaces the minimal sufficient set of visited pages that collectively support the agent’s answer, balancing completeness with cognitive load. Second, HANSEL *preserves actionable state*: a URL alone is insufficient if the user lands on a default page and must reapply filters or search queries to find the relevant information. HANSEL reconstructs the page state the agent encountered (e.g., scrolling, applying filters, filling in search queries, clicking specific elements), so users can inspect the evidence without reapplying these actions manually, reducing the cost of interaction. Finally, HANSEL *makes errors visible*: when the agent’s answer cannot be traced to any visited page or is potentially incorrect, HANSEL makes it easy to detect by providing multiple views that are carefully designed to help users follow agents’ reasoning traces.

We evaluated HANSEL with both a technical evaluation and a user study. Our technical evaluation revealed that HANSEL accurately identifies core evidence pages from agent trajectories, achieving 83.7% precision and 88.8% recall against ground-truth annotations across 45 tasks from AssistantBench [52] and Online-Mind2Web [49]. In a controlled user study with 14 participants, we further demonstrated that HANSEL showed a trend toward higher task completion accuracy, and significantly reduced completion time compared to the baseline. Participants also reported significantly lower effort in their judgments when using HANSEL. Finally, participants rated HANSEL as useful and found its key features helpful.

In summary, we contribute:

- A formulation of the agent evidence extraction problem,
- An interactive verification interface that enables users to engage directly with evidence, rather than passively consuming agent explanations,
- Empirical evidence from a user study demonstrating that interactive evidence presentation leads to more efficient verification.

2 Background & Related work

2.1 LLM-based Web Agent

With the advancement of LLM capabilities in web navigation [30, 48], web navigation tasks have shifted from reinforcement learning (RL) tasks [18, 39] toward LLM-based web agents [20, 31, 53]. These agents show strong performance on benchmarks such as Mind2Web [10] and WebArena [55], and are increasingly useful for completing real-world tasks on behalf of users. However, web agents still face challenges in real-world web tasks. Agents frequently make grounding errors, failing to correctly identify and interact with the intended page elements [10, 50]. They also get stuck in loops and fail to recover in long-horizon tasks [8, 52]. Even when agents correctly understand the task and perform reasonable actions, they often miss crucial details and lose track of the original task objectives [25]. Real-world websites introduce additional challenges, such as unpredictable bot detection mechanisms, which can break automation and require human intervention [49]. Despite these failures, agents can still produce a result even when errors are hidden in intermediate steps and are not visible in the final response, motivating us to design HANSEL to help users verify whether the agent actually completed the task correctly.

2.2 Verification of AI Agents

Previous work has explored approaches that allow users to monitor and intervene in agent execution in real-time [21, 29, 37]. Cocoa [14] extends this approach to co-planning and interactive debugging. However, users cannot be expected to monitor agent behavior at all times during execution [7, 29], and even when users proactively collaborate with the agent, errors in task outcomes remain [19, 44]. Post-hoc support is therefore needed to help users verify agent behavior after execution. One common way to support post-hoc verification is to summarize the agent’s reasoning and execution process [40, 51]. However, these summaries are often not faithful to the agent’s actual process and may contain inaccurate information [23, 27, 43], and this unfaithfulness persists across multiple model families and even in highly capable models [3, 42]. When these summaries appear coherent and plausible, they tend to increase users’ reliance on incorrect answers rather than help users detect them [2, 6, 36, 45]. To genuinely understand and verify agent behavior, users need access to the actual agent execution trajectory. Researchers have explored different ways to expose trajectories effectively. Earlier work made intermediate reasoning steps visible and editable, helping users verify and intervene in the reasoning process more efficiently [47]. As agentic systems have become more complex, recent work has introduced tools for visualizing agent execution trajectories across different domains. OpenHands Trajectory Visualizer [34] renders raw step-by-step execution logs of coding agents into structured views. AgentDiagnose [35] provides a visualization dashboard for web agent trajectories, annotating each step with automatic evaluation scores to assess agent behavior. For multi-agent systems, AGDebugger [12] presents agent message histories with an overview visualization for navigating long conversations, and XAgen [46] parses execution logs into interactive step-by-step flowcharts. Despite improving the understanding of agent execution, these approaches are primarily designed for developers debugging agent workflows and still require people to follow the trajectory step by step to identify relevant information. We further discuss how existing web agent systems fall short in Section 3.2. In our work, we design HANSEL to support post-hoc verification for web agents: rather than visualizing the full execution trajectory, HANSEL extracts and presents only the evidence directly relevant to verifying the task outcome, reducing the verification burden for end users.

3 Formative Analysis and Design Goals

3.1 Empirical Analysis of Agent Trajectories

To understand the scale and composition of agent trajectories and characterize potential information overload challenges [13], we manually analyzed 45 tasks from AssistantBench [52] and Online-Mind2Web [49]. AssistantBench is a benchmark of realistic, time-consuming information-seeking tasks on the open web, each with a ground-truth answer. It provides a development set with 33 tasks for experimentation and analysis. Online-Mind2Web [49] is a benchmark of 300 realistic web navigation tasks spanning 136 websites, covering a broad range of procedural goals. We collected the actual

agent trajectory logs from HAL [22], which evaluates AssistantBench and Online-Mind2Web using browser-use [5], a Python-based Playwright browser automation agent. We selected the top-performing model for each benchmark at the time of our study based on the reported accuracy in HAL. For AssistantBench (33 tasks), we used trajectories for these 33 tasks generated with o3-medium (38.8% accuracy) in HAL. We removed tasks where the agent failed to produce an answer, and those where no web interaction was performed (i.e., the agent directly produced an answer in the first step without any web actions), resulting in a final set of 22 tasks. For Online-Mind2Web (300 tasks), we used trajectories generated with Claude Sonnet 4 (40.0% accuracy). We excluded open-ended tasks that did not have a single verifiable ground-truth answer (e.g., “Compare Apple Watches and learn more about the Ultra version on Apple”) and randomly sampled 23 tasks from the remaining set of 140 using a fixed random seed. This resulted in a combined analysis set of 45 tasks (22 from AssistantBench and 23 from Online-Mind2Web).

For each task, the first author examined the full trajectory and labeled each step as an evidence step or a non-evidence step. We annotated the trajectories at two levels: steps and pages. A step is an atomic observation-reasoning-action unit, consisting of an observation of the current page state, reasoning about the next goal or plan, and action execution, followed by immediate environmental feedback. A step was labeled as an evidence step if it contributed to the reasoning process leading to the final answer. Steps corresponding to failed attempts, navigation overhead (e.g., loading homepages, scrolling), blocked pages (e.g., login walls, bot protection), or repeated retries without new information were excluded. A page is defined as a unique URL visit. Consecutive steps sharing the same URL are grouped into a single page, while non-consecutive visits to the same URL are counted as separate pages. For example, in a trajectory page1 → page2 → page3 → page1, the two page1 visits are counted as two separate pages. A page was labeled as an evidence page if it contained at least one evidence step. To assess annotation reliability, another independent annotator familiar with web agent systems labeled the full dataset of 592 steps and 271 pages from 45 trajectories. Inter-rater agreement reached Cohen’s Kappa of 0.803 at the step level and 0.878 at the page level, indicating good agreement. Disagreements were resolved through discussion between the two annotators to produce the final results.

Table 1: Trajectory statistics across datasets.

	AssistantBench	Online-Mind2Web	Overall
Tasks	22	23	45
Total steps	204	388	592
Total pages	146	125	271
Evidence steps	44 (21.57%)	106 (27.32%)	150 (25.34%)
Evidence pages	36 (24.66%)	62 (49.60%)	98 (36.16%)

Across the 45 trajectories, agents executed an average of 13.16 steps and visited 6.02 web pages per task (Table 1), yet only 3.33

steps (25.34%) and 2.18 web pages (36.16%) on average contributed to the final answer. In 28 of 45 tasks (62.22%), the final answers were directly supported by fewer than 5 steps and no more than 3 evidence pages. In 16 out of 45 tasks (35.56%), the final answer only required a single page visit, while the agent visited 5.19 pages on average. The remaining steps corresponded to navigation overhead (e.g., loading homepages, scrolling), abandoned search paths, or failed interactions. Although Online-Mind2Web had longer trajectories than AssistantBench (16.87 vs. 9.27 steps per task), the fraction of steps and pages that contributed to the final answer remained small in both. The page-level evidence ratio in Online-Mind2Web appears higher because many within-page actions are irrelevant to the final answer, even when they occur on an evidence page. This analysis empirically validates that the small subset of steps that matter for verification is often buried within a much larger volume of irrelevant information.

3.2 Why current mechanisms fall short

While modern web agents offer several forms of transparency to help users understand how an answer was produced, most of the current mechanisms fall short. In this subsection, we discuss how current agentic systems convey this information, and why they may not be sufficient for users to grasp what the agent actually did.

3.2.1 Execution logs. Systems built on ReAct-style reasoning [51] provide the agent’s complete action-and-reasoning trace. While comprehensive, these logs can be overwhelming: all steps, including navigation overhead, abandoned search paths, and failed interactions, are displayed without curation, making it difficult for users to identify what to focus on. For example, in a shopping task, the agent may repeatedly refine and retry searches while switching between listing and detail pages without making progress, with each attempt logged with detailed actions (e.g., type, click, scroll) and reasoning traces (e.g., “Failed”, “I need to ...”), even though none of them contribute to the final answer.

3.2.2 Source page links. Other systems provide links to the pages an agent visited. In such systems, agents present links to sources and related content that correspond to parts of the generated response. For example, Gemini provides source links either inline or through a side panel, allowing users to access related webpages for further inspection [17]. However, for more complex web tasks, this approach loses important context, such as the page state. Clicking a link loads the page’s current default state, not the state the agent encountered. If the agent sorted results by price and filtered by location, the user who follows the link sees an unfiltered, default-sorted page and must manually reconstruct the agent’s search context to verify the answer.

3.2.3 Screenshots. Some systems capture static snapshots of pages that the agent visited, offering a more intuitive view of the agent’s browsing process than raw logs. For example, after completing a task, OpenAI’s Operator [33] presents its response with embedded screenshots of key webpages visited during browsing. However, screenshots are non-interactive: users cannot scroll to see content below the fold, check whether a filter was correctly applied, or inspect elements that were off-screen when the snapshot was taken. An agent error involving a cheaper option listed further down

the page, or an incorrectly set search radius on a map, would be invisible in a static image.

3.2.4 Reasoning summaries. Many agents such as WebGPT [31] produce natural-language summaries of their process. The core limitation is that these summaries can rationalize incorrect answers as fluently as correct ones [27, 43]. For example, a summary may describe how the agent successfully identified a product satisfying all constraints, while the agent in fact failed to apply the correct filters and missed a qualifying option. Users can only verify the summary by actually following the process themselves. These plausible-sounding explanations may increase over-reliance by creating an illusion that verification has already been performed [15].

3.2.5 The Gap. The common gap across all four current mechanisms is that verification is still treated as a passive reading task; the user is a consumer of logs, images, links, or text, and the verification burden is put onto users. However, effective verification is fundamentally interactive; users need to re-sort, probe filters, and cross-check directly [38, 54].

3.3 Design Goals

To fill this gap, we extract the pages from an agent’s trajectory that contribute to the final answer and organize them as interactive evidence. The design of HANSEL is guided by the following goals.

- **DG.1: Surface a Minimal Sufficient Evidence Set:** Users can verify the agent’s answer more quickly with a concise set of evidence pages that support the answer.
- **DG.2: Enable Direct Evidence Access:** Users can directly inspect the evidence through interactive pages reconstructed by HANSEL with critical state preserved. If the agent fails, users can continue the task from the relevant page state.
- **DG.3: Improve Evidence Gap Awareness.** Users can recognize when an agent’s claims are unverifiable or potentially incorrect.

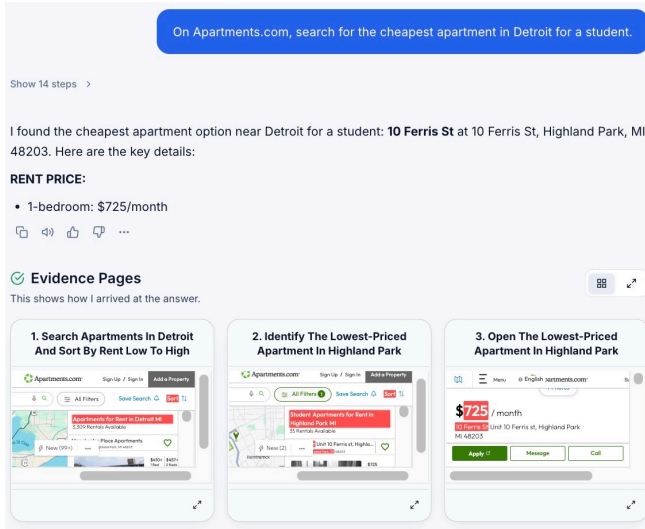
4 HANSEL

We designed and implemented HANSEL to support *efficient agent verification through interactive verification*.

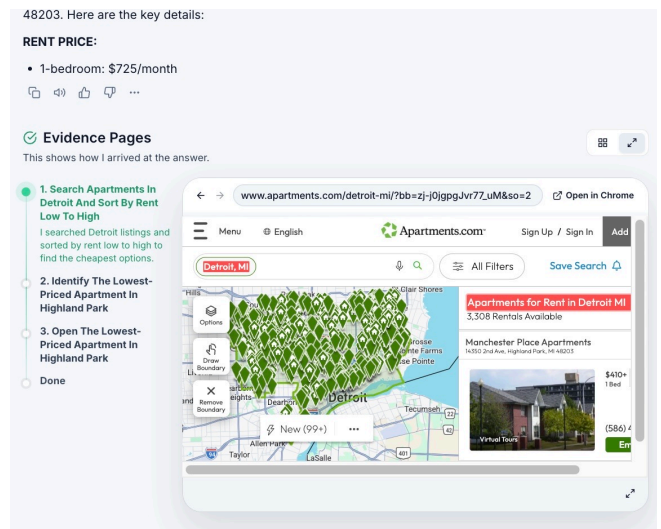
4.1 DG1: Evidence Page Extraction

In our empirical analysis of agent trajectories (Section 3.1), only a small fraction of pages directly support the final answer, suggesting that presenting this subset of evidence pages is sufficient to enable users to verify the correctness of an agent’s output.

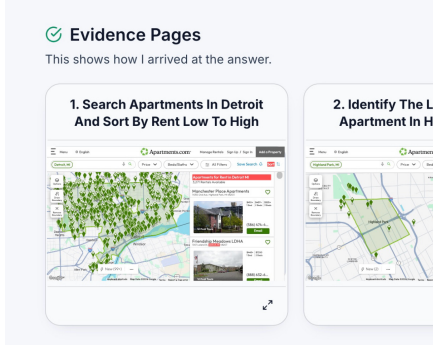
We build an evidence extraction pipeline that automatically extracts evidence pages and evidence snippets from the agent’s trajectory log to surface a minimal sufficient evidence set (DG1). Following the annotation scheme introduced in Section 3.1, an *evidence page* is a page that contains at least one step whose content directly contributes to the reasoning toward the final answer. This excludes pages that correspond to failed attempts, abandoned branches, or navigation steps that do not contribute useful information. Beyond identifying *evidence pages*, we further extract *evidence snippets*: the specific text within an evidence page that the user needs to attend to. We extract this more granular information because a single page



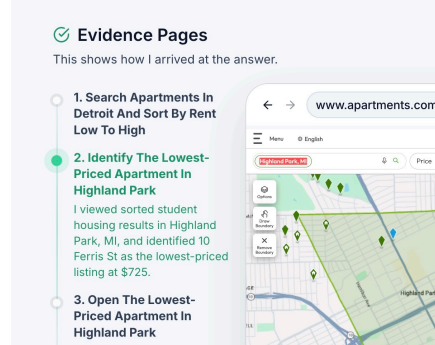
(a) Overview of Evidence Pages



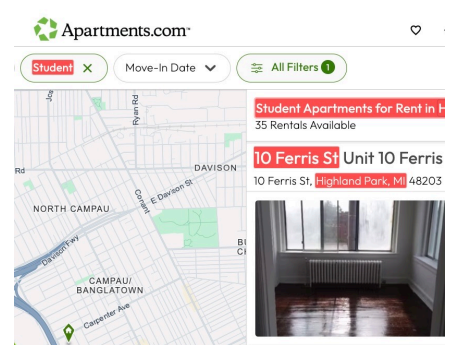
(b) Sequential Inspection of Evidence Pages



(c) Evidence Pages with Concise Titles



(d) Navigation of Evidence Pages



(e) Interactive Webpage with Highlighted Evidence

Figure 2: HANSEL identifies a subset of visited pages from web agent trajectories and presents them as interactive evidence pages. For example, when asked “Search for the cheapest apartment in Detroit for a student on Apartments.com.”, HANSEL first extracts three evidence pages out of 14 steps and presents them as a global overview under the agent’s response (a). Each evidence page is accompanied by a concise title that summarizes the agent’s actions (c), allowing users to quickly understand how the agent arrived at the answer. Users can then move to (b) to inspect these pages in sequence, and navigate across them (d). HANSEL provides an embedded live, interactive webpage for each evidence page, with key evidence highlighted in red (e). During inspection, users can verify highlighted evidence, such as location and filter settings, and modify them directly on the webpage within the interface.

may contain multiple steps, only some of which are relevant. For example, within a single evidence page, the agent may type in search boxes to get autocomplete suggestions, scroll to discover additional content, apply filters to narrow down options, and change sorting criteria. If only the sorting criteria informed the final answer, the evidence snippet would consist of only the content related to the sorting criteria. We now introduce the two components of this extraction pipeline.

4.1.1 Execution Log Standardization. Raw execution logs contain rich information about what the agent did, including the full sequence of agent actions, intermediate states, and environment responses, so we treat them as the input to HANSEL. However, raw logs are verbose, with much of the execution metadata irrelevant to evidence extraction, and heterogeneous across frameworks. For example, browser-use scaffold logs a tree structure of LangChain operations with entries $\langle id, parent_id, op_name, t_{start}, t_{end} \rangle$. In contrast, the HAL Generalist Agent scaffold organizes reasoning steps as a linear sequence $[(role_1, content_1), \dots]$, with no analogous tree structure. Despite incompatible architectures, both scaffolds implement the same Observation-Reasoning-Action cycle. We therefore use a

standardized trajectory representation (*observation, reasoning, action*) to support evidence extraction across agents. We represent an agent trajectory as an ordered sequence of interaction steps produced while solving a task. Each step records the agent’s observation, reasoning, and action. Formally, a trajectory can be written as

$$T = [(o_1, r_1, a_1), (o_2, r_2, a_2), \dots, (o_n, r_n, a_n)]$$

The observation o_i captures the agent’s environmental input at step i , typically a representation of the current page state (e.g., raw HTML, a DOM snapshot, an AXTree view), and may also include the result of the previous action (e.g., an error message or confirmation). For example, one trajectory records the following page snapshot as an observation: `[0]<a title="" />, [1]<div />, [2]<textarea ...>`. The reasoning r_i describes the agent’s thought about the current state, including how it evaluates the previous action results, interprets the observation, and decides the next action. For example, when CAPTCHA blocks access, the reasoning may state that the previous goal failed due to the CAPTCHA page and decides to switch to an alternative search path. The action a_i is one of a set of predefined browser operations supported by the scaffold, such as click, type, and search, each executed with explicit parameters. For example, a click action specifies a target element index (e.g., `click(index=24)`), while a search action specifies a query string (e.g., `search(query="Detroit apartments")`).

This standardized representation abstracts low-level execution details specific to each scaffold (e.g., LangChain operation trees) while preserving the information necessary for evidence extraction and answer verification.

4.1.2 LLM-Based Evidence Extraction. Given a trajectory $T = [(o_1, r_1, a_1), \dots, (o_n, r_n, a_n)]$, the first goal of HANSEL is to extract all evidence pages and identify all evidence snippets on each page that directly support the final answer. We use an LLM to identify (i) which visited pages are evidence pages, and (ii) which specific content on each page should be included as evidence snippets. We provide the LLM with: (1) the user query, (2) the final answer, and (3) the steps (o_i, r_i, a_i) extracted from the logs. We instruct the LLM to group steps into page-level units following the same convention as our annotation (Section 3.1). Consecutive steps sharing the same URL form a single page, while non-consecutive visits to the same URL are treated as separate pages. The LLM outputs a list of evidence pages in the order they were visited. For each page, the output contains a brief title and a more detailed description of the agent’s actions on this page, as well as all evidence snippets identified on that page.

To ensure that the extracted evidence is concise and relevant, we design a set of prompt rules. Since agents often follow multiple different plans given a single task, or revise their plans during execution if their original plan fails, we instruct the LLM to discard abandoned or unsuccessful plans, and report only the final successful reasoning paths. We also instruct the LLM to remove blocked pages (e.g., connection errors, login gating, bot protection), and intermediate pages resulting from redirects, as these do not provide usable information for answering the task.

4.2 DG2: Interactive Evidence Verification

During manual annotation, we observed that several failures were not major reasoning errors but small interaction mismatches (e.g., an incorrect sort option). For example, in the task “Find the 12 Monkeys community and find the latest post mentioning James Cole on Reddit”, the agent successfully found the community and searched for James Cole, but did not change the sort order from Relevance to New. These minor errors are easy to fix, but they become costly when users must reopen pages externally and repeatedly switch back to the task interface to check.

This motivates our design to render each evidence page as a live, interactive webpage embedded directly within HANSEL, where users can easily inspect and correct errors when necessary. Users can directly interact with the embedded page, including clicking, scrolling, inputting, or navigating, without leaving the HANSEL interface. Each evidence page is initialized in a reconstructed state that mirrors what the agent encountered (e.g., with filters applied or search queries pre-filled). Specifically, HANSEL parses the trajectory to extract the actions performed on that page, and replays them, so that users do not need to reapply these actions manually. HANSEL also highlights the extracted evidence snippets directly on the live page, helping users quickly locate key information without reading the entire page.

4.3 DG3: Reasoning Visualization

To assist users in inspecting the evidence page and recognizing when an agent’s claims are unverifiable or potentially incorrect, HANSEL provides multiple ways to display the evidence pages and the snippets. HANSEL presents evidence pages in two complementary layouts: grid view and carousel view. Users can switch between these layouts depending on whether they want a global overview or a more detailed, step-by-step inspection.

The grid view displays multiple evidence pages simultaneously, arranged in order based on their timestamp in the agent’s trajectory (Figure 2a). This view is designed to help users easily spot errors in the high-level reasoning. For example, if the agent simply provided an answer to the task based on its training time memory, there will be no evidence page or snippet to show. In this view, each evidence page is associated with a concise, descriptive title that summarizes the agent’s action on that page, enabling users to quickly understand what the agent did. In Figure 2a, the three evidence pages are presented. Users can expand any page to inspect it in detail and interact with it directly.

The carousel view organizes evidence pages based on their timestamps and presents them one by one, allowing users to inspect pages in order (Figure 2b). In the left panel, HANSEL provides a timeline with a short title and a detailed description for each page, allowing users to navigate between evidence pages. In the right panel, HANSEL embeds a larger browser window that displays the reconstructed page with highlighted evidence, allowing users to inspect and interact with it directly. Users can also expand the page to full screen or open it in an external browser.

On every page, HANSEL highlights the extracted evidence snippets directly on the live page. For example, in Figure 2e, the student filter (Student in the top-left corner), the apartment search location (Highland Park, MI), and the reported final answer (10 Ferris

Table 2: Page-level evidence extraction quality and snippet-level precision across two benchmarks.

Dataset	#Tasks	Page-level		Snippet-level	
		Precision	Recall	F1	Precision
AssistantBench	22	0.821	0.889	0.853	0.838
Online-Mind2Web	23	0.846	0.887	0.866	0.902
Overall	45	0.837	0.888	0.861	0.887

St) were highlighted. This is to help users directly locate the evidence snippets that contributed to the agent’s answer without exploring the entire page to find them.

4.4 System Implementation

For the evidence extraction, we used a Python-based pipeline. HANSEL first converts the raw trajectory log into a standardized sequence of steps, where each step contains the agent’s observation, reasoning, action, and current URL. Given the user query, the final answer, and the standardized trajectory steps, HANSEL uses an LLM to identify which visited pages directly support the final answer and which specific content on those pages should be highlighted as evidence. We use GPT-5.4 with a temperature of 0.7. The extracted results are stored as structured JSON objects, including page-level summaries and snippet-level evidence spans. The pipeline also parses the trajectory to recover the actions performed on each evidence page (e.g., scroll, input, click), which are later replayed in the interface to reconstruct each page into the state the agent reached during task execution.

The interface of HANSEL was implemented as a desktop application using React and Electron [11]. To enable users to view highlighted evidence and interact with live webpages in HANSEL, we initially explored using iframes to embed evidence pages. However, iframes are often blocked by websites due to security restrictions. We instead use Electron’s BrowserView to embed a live, interactive browser instance, and HANSEL replays scripted actions such as waiting, clicking, scrolling, and selecting filters to reconstruct each page into the state the agent reached. HANSEL then highlights the evidence snippets identified by the LLM directly on the page by injecting CSS styles into the embedded browser at render time.

5 Technical Evaluation

Before testing HANSEL with human participants to evaluate its usefulness in verification, we first tested whether the core evidence extraction component of HANSEL provides accurate evidence. We used the agent trajectory dataset we created in Section 3.1, containing 22 tasks from AssistantBench and 23 from Online-Mind2Web. From this evaluation, we measured page-level extraction quality and snippet-level precision.

Page-level extraction For each task, we compared the extracted evidence-page set with the manually annotated ground-truth set from Section 3.1. We then computed Precision, Recall, and F1 from these aggregated counts (Table 2).

Across 45 tasks, our method achieved an overall F1 score of 0.861 for identifying evidence pages supporting the final answer.

HANSEL reduced the information volume from 271 trajectory pages to 104 extracted evidence pages (61.6% reduction).

Snippet-level precision For snippet-level precision evaluation, two annotators manually reviewed the evidence snippets extracted by the LLM on correctly extracted evidence pages. An evidence page can contain multiple evidence snippets (e.g., filters and facts), and an evidence snippet was labeled as valid only if it was visually observable on the webpage and supported the final answer. Inter-rater agreement reached Cohen’s $\kappa = 0.742$ at the snippet level, and disagreements were resolved through discussion to produce the final labels.

In total, our method produced 159 evidence snippets (37 on AssistantBench and 122 on Online-Mind2Web). Of these, 31 out of 37 snippets were judged valid on AssistantBench (0.838), and 110 out of 122 were judged valid on Online-Mind2Web (0.902), yielding an overall snippet-level precision of 0.887 (141/159).

6 User Evaluation

To evaluate the impact of HANSEL on verifying AI agents’ answers and finding correct answers when necessary, we conducted a user study with participants. With the user study, we answered the following research questions:

- RQ1: Does HANSEL improve users’ ability to correctly verify web agent task outputs?
- RQ2: Does HANSEL reduce the time and effort required in verifying web agent task outputs?
- RQ3: How do users perceive and use HANSEL when verifying web agent task outputs?

In this study, we used a within-subject design with two conditions: (A) HANSEL and (B) the baseline, a standard conversational agent interface that displays the user query, the agent’s final response, and a set of related source pages. Both conditions included a step-by-step action trajectory. The study procedure and participant recruitment complied with our institution’s IRB requirements.

6.1 Participants

We recruited participants from a large research university, through social media and university forums. Participants were pre-screened to ensure they had used AI tools and were proficient in English. Out of 14 participants, 6 identified as male and 8 identified as female. Participants came from diverse academic backgrounds. 7 participants were from computing and engineering fields, 3 from business and economics, 2 from law, 1 from cognitive science, and 1 from communication and journalism. In terms of their familiarity with AI tools, all participants mentioned that they had used conversational AI tools, but 8 participants (57%) reported that they did not use the agent mode. 2 participants (14%) reported only modest proficiency in completing web-based tasks.

6.2 Tasks

We selected eight tasks to simulate scenarios in which users interact with web agents to assist with everyday tasks, from the manually annotated data in Section 3.1. Many AssistantBench trajectories were not suitable for the user study because the agent failed to make meaningful progress or did not produce a usable task outcome. In

	Task 1	Task 2	Task 3	Task 4	Task 5	Task 6	Task 7	Task 8
Agent's answer	C	W	W	C	C	W	W	C
Group A (n=7)	HANSEL	Baseline	HANSEL	Baseline	HANSEL	Baseline	HANSEL	Baseline
Group B (n=7)	Baseline	HANSEL	Baseline	HANSEL	Baseline	HANSEL	Baseline	HANSEL

Figure 3: Experiment design across two groups. Participants alternated between HANSEL and the baseline across eight tasks, with agent answers following a fixed correct/wrong sequence (C = correct, W = wrong). Tasks were paired by type: pricing calculation (Tasks 1 & 4), sorting/filtering (Tasks 2 & 7), selection under constraints (Tasks 3 & 6), and information retrieval (Tasks 5 & 8).

such cases, there was little for participants to verify; completing the task would have required starting from scratch, which would confound our comparison of verification effectiveness. Therefore, we selected 2 tasks from AssistantBench and 6 from Online-Mind2Web. Many AssistantBench trajectories were not suitable for the user study because the agent failed or only partially completed the task, leaving little verifiable output for participants to assess. Solving these tasks from scratch are too time-consuming for our study. Therefore, we selected 2 tasks from AssistantBench and 6 from Online-Mind2Web. The selected tasks involve multi-step information foraging, such as “How much would I save by getting an annual membership for my family (2 adults, 1 kid age 5, 1 kid age 2) living in Washington State for the Seattle Children’s Museum, compared to buying daily tickets, if we visit 4 times in a year?” We selected two tasks for each of four task types (e.g., pricing calculation, information retrieval, sorting/filtering, and selection under constraints), so that one task of each type could be assigned to the baseline condition and the other to the HANSEL condition.

To test the interface’s impact on participants’ verification of agent responses, we selected 4 tasks in which agents returned correct answers and 4 in which they returned wrong answers. Among the 4 agent responses with errors, 2 involved incorrect filtering or sorting (e.g., failing to select the post with the most replies or the latest post), and 2 involved incorrect judgments based on webpage content (e.g., selecting a product that was not the lowest priced, or searching in the wrong location, such as Highland instead of Detroit).

During the study, participants were assigned to use the two conditions alternately across eight tasks (Figure 3). To avoid predictable patterns in task outcomes, we randomly assigned the tasks with correct and incorrect agent answers. To minimize the noise coming from the order of the tasks, we kept the task order fixed across participants, and only divided participants into two groups who started with HANSEL or the baseline.

6.3 Study Protocol

We conducted the study through a video conferencing tool and in person, and each session took about 60-70 minutes. At the beginning, we provided a short introduction of the study and obtained

participants’ consent. The participants then completed a pre-study survey to report their background. We then introduced the study setting and task procedure. In this study, participants interacted with the interface via remote control. Participants were asked to complete 8 web-based tasks with a time limit of 5 minutes per task. For each task, we provided the task description and the agent’s response, and we asked participants to submit the correct answer once they were confident. They could decide whether to accept the agent’s response or revise it if they find issues. If needed, participants could use external search engines during the task. Before the main tasks, we presented a tutorial for the two interface conditions. Participants were shown both interfaces with a short demonstration of their features. We then started the main study session. For each task, we recorded task start and end times, participants’ final answers, and their interactions with the interface. After each task, participants reported their confidence in the answer and the perceived effort using 7-point Likert-scale questions. After completing all eight tasks, participants completed a post-study survey. The post-study survey included questions assessing participants’ perceptions of the two interfaces and the usefulness of key interface features, both using 5-point Likert-scale questions. At the end of the study, we conducted a semi-structured interview to collect participants’ solution strategies, feedback on both interfaces, and overall experience.

6.3.1 Data Collection. We collected both quantitative and qualitative data during the study. Before the study, participants completed a recruitment form and a pre-study survey. For each task, we recorded (1) completion time (from start and end timestamps), (2) answer correctness (correct/incorrect), and (3) self-reported confidence and effort ratings collected in the post-task survey. After all tasks, participants completed a post-study survey that included overall interface preference, as well as paired comparisons between HANSEL and the baseline and feature-level usefulness ratings. All sessions were recorded, and we transcribed participants’ responses from the interviews.

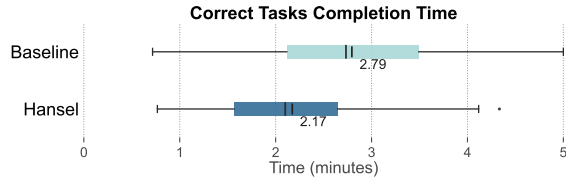


Figure 4: Completion time (minutes) by interface (the baseline vs. HANSEL) across tasks in which participants submitted a correct answer

7 Results

7.1 RQ1: Accuracy

We fit a logistic mixed-effects model predicting submission accuracy, with a variable `uses_hansel` indicating the condition (baseline vs. HANSEL), task as a fixed effect, and participant as a random intercept. We included task as a fixed effect to control for inherent differences in difficulty across the eight tasks. Participant was modeled as a random intercept to account for individual differences, as observations from the same participant are not independent in our within-subjects design. Accuracy was high in both conditions, with 75.0% for the baseline and 82.14% for HANSEL. On tasks where the agent’s answer was correct, accuracy was 89.3% for the baseline and 96.4% for HANSEL; on tasks where the agent’s answer was incorrect, accuracy was 60.7% for the baseline and 67.9% for HANSEL. Across both conditions, accuracy was approximately 7 percentage points higher with HANSEL, suggesting a trend that participants submitted more correct answers when using HANSEL.

However, we found no significant difference between conditions across all tasks ($\beta = 0.71$, $SE = 0.67$, $z = 1.06$, $p = .29$). To better understand how participants verified answers across conditions, we analyzed their verification behaviors from interaction logs. Participants using the baseline frequently re-executed parts of the task themselves (e.g., reopening webpages, rerunning searches, and manually applying filters) in 66% of tasks, whereas participants using HANSEL did so in only 9% of tasks, and relied more on the extracted evidence. This suggests that participants in the baseline condition could often reach correct answers by redoing parts of the task from scratch, but doing so required additional effort. We further examine this difference in verification effort in RQ2. We also observed that HANSEL users accepted the wrong answer in 32.1% of cases when the agent’s answer was incorrect. We discuss this trust-related limitation and its implications further in Section 8.2.

7.2 RQ2: Effort

We analyzed task completion time only for tasks in which participants submitted a correct answer ($N = 88$), as completion time for incorrect submissions can be difficult to interpret. A short time may indicate accepting the agent’s output without careful verification, whereas a long time may indicate unsuccessful verification. Using a linear mixed-effects model with the same fixed and random effects, participants completed tasks significantly faster ($\beta = -43.20$, $SE = 10.03$, $t = -4.31$, $p < .001$) when using HANSEL ($\mu = 130.4$ sec, $\sigma = 53.2$ sec) than when using the baseline ($\mu = 167.7$ sec, $\sigma = 61.0$ sec) (Figure 4). In the time model, three tasks showed

significant differences relative to Task 1 (Task 3: $p = .008$, Task 7: $p = .005$, Task 8: $p = .018$), indicating non-trivial between-task differences in completion time.

In addition to completion time, participants rated their perceived effort after each task on a 7-point Likert scale. To compare their perceived effort levels between the baseline and the HANSEL conditions, we used a Wilcoxon signed-rank test. Given that we used the within-subjects design, we aggregated each participant’s effort ratings per condition using median ratings to perform a paired test. For this comparison, we also only used the tasks where participants submitted a correct answer ($N = 88$) for the same reason. Participants reported significantly lower effort ($V = 7$, $p = .012$, $r = .67$), when using HANSEL (average median = 2.89) compared to the baseline (average median = 4.39). Together with the significant reduction in task completion time, this indicates that HANSEL reduces both the objective and subjective cost of verification.

7.3 RQ3: Perceived Usefulness

All 14 participants reported that they preferred HANSEL over the baseline interface and agreed that HANSEL made it easier to verify whether the agent’s answer was correct.

Interface Usefulness Ratings In the post-study survey, participants rated the two interfaces across five dimensions: *Understanding* (“helped me understand how the agent arrived at the answer”), *Verification* (“made it easy to verify the agent’s response”), *Error Identification* (“helped me identify potential problems in the agent’s response”), *Correction* (“helped me revise the response when it was wrong”), and *Usability* (“easy to learn and use”).

We compared the participant-reported 5-point Likert scale ratings using participant-level paired, two-sided Wilcoxon signed-rank tests for each dimension. Participants consistently rated HANSEL higher than the baseline across all five dimensions. Participants rated HANSEL to be significantly easier to use ($\mu = 2.93 \rightarrow 4.71$, $p = .0045$) and to verify the agent’s response ($\mu = 2.43 \rightarrow 4.43$, $p = .0013$). Using HANSEL, participants found it significantly easier to understand how the agent arrived at the answer ($3.07 \rightarrow 4.50$, $p = .0017$) and revise the response when it was wrong ($2.50 \rightarrow 3.93$, $p = .0045$). Participants also reported that HANSEL significantly improved their ability to identify potential problems in the agent’s response ($2.57 \rightarrow 4.00$, $p = .0127$) (Figure 5).

Feature Usefulness Ratings Participants rated HANSEL’s evidence-grounding capabilities highly. Participants rated features that support direct interaction with evidence the highest, such as *Highlighted Evidence* (4.79/5), *Direct Interaction with Pages* (4.64/5), and *Evidence Pages with Titles* (4.43/5), with 93%, 100%, and 100% of responses at 4 or above, respectively. In contrast, participants gave comparatively lower ratings to *Source Links* (3.14/5; 43% ≥ 4) and *Step-by-step Trajectory* (3.36/5; 64% ≥ 4) (Figure 5).

7.4 Qualitative Analysis

The first author coded the number of unique pages visited per task from recordings, counting each distinct page visited within the HANSEL interface or on external websites. Two authors then conducted a thematic analysis [9] of the interview transcripts, which included participants’ task walk-throughs and open-ended responses.

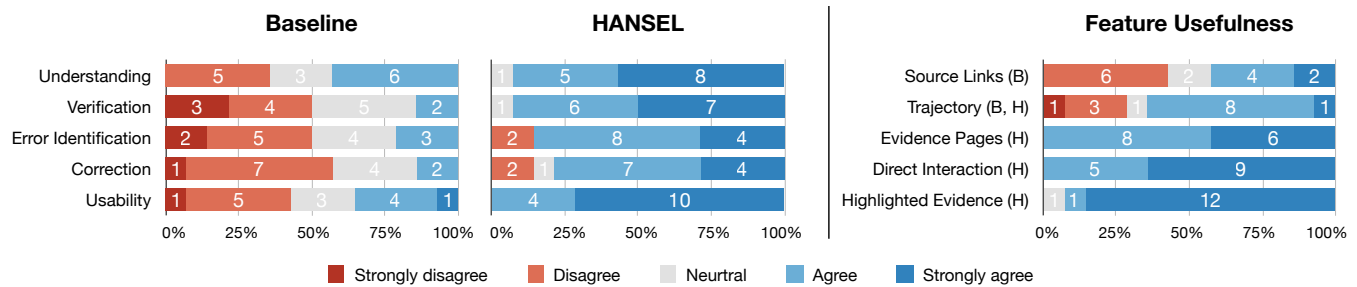


Figure 5: (Left) Summary of responses to the post-study survey comparing the baseline and HANSEL across five questions. (Right) Distribution of participant ratings for the usefulness of features

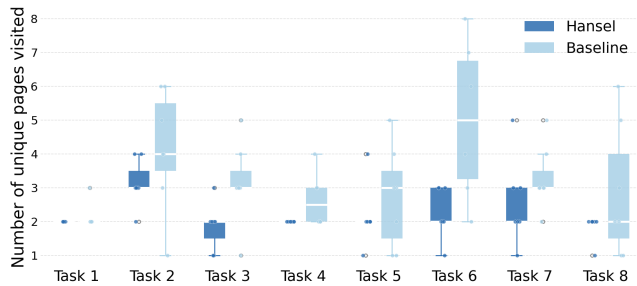


Figure 6: Number of unique pages visited per task (HANSEL vs. the baseline)

One author performed initial open coding, and the codes were refined through discussion with another author to derive key themes. Differences in interpretation were resolved through discussion until consensus was reached.

Participants used different verification strategies across interfaces. Across all eight tasks, participants visited fewer web pages when using HANSEL than when using the baseline (Figure 6). In the HANSEL condition, participants consistently browsed evidence pages in order directly within the interface, which surfaced the agent’s navigation path without requiring them to reconstruct it. “If there was the interface with the evidence page, I would just open up the evidence page from there.” (P6) When the agent’s answer was incorrect, many participants noticed the error within the interface and corrected it directly on the evidence page without navigating away or restarting the task. For example, in Task 6 (the task shown in Figure 2), participants using HANSEL treated the second evidence page as a sufficient entry point for correction, leading to notably fewer external pages visited compared to the baseline. In the baseline condition, some participants visited no external pages at all for certain tasks, accepting the agent’s answer without further verification. “I forgot I can check the source link in this task.” (P4) Others retraced the agent’s search path from scratch, navigating to entry points such as flightaware.com or bestbuy.com to verify the result manually. In some cases, participants relied on the step-by-step execution logs to inspect the agent’s reasoning and locate relevant pages. “I want to double check, so I open the steps from the agent and find another website.” (P8) “I want to know what the AFC East Division is, so I go to the steps to find a related link.” (P9)

Participants’ verification behavior was shaped by their familiarity with the task domain. When participants felt familiar with a domain or website, they were more capable of detecting errors in the agent’s response and correcting them. “I know Flightaware is a popular platform, so the post with most replies couldn’t only have 54 replies.” (P1) “This is an interface I am very familiar with, so I thought it should be sorted by latest rather than using the relevance filter.” (P9) In contrast, participants described unfamiliar websites as harder to navigate and more effortful to verify. “Like the Steam pages, sometimes even when I go to the source, I do not really know where to navigate.” (P12) In such cases, some participants deferred to the agent even when they were not confident in its output, and were sometimes misled by the agent. “I trust AI because I have very little knowledge about the NFL.” (P11) In HANSEL, evidence pages partially compensated for low familiarity by surfacing intermediate pages from the agent’s navigation path, giving participants more context to evaluate the answer.

Highlights guided verification but were not always visually distinguishable. Most participants reported that highlights helped them locate relevant evidence and perform key verification actions more quickly, particularly on unfamiliar websites. “With highlights, your attention is naturally drawn to the important parts.” (P12) However, highlights were not always interpreted correctly when they blended with the styling of the underlying webpage. “I thought it was part of the webpage... it was mixed with the purple.” (P4) To improve the effectiveness of highlights, future designs could adapt highlight styles to different webpage designs and use visually distinct styles for different types of highlights (e.g., textual evidence, filters, search queries), helping users better distinguish their roles and interpret them correctly.

Participants developed divergent trust orientations toward the agent. After encountering errors earlier in the study, some participants approached the agent’s output with skepticism. “I assume the agent answer was incorrect and still believe in myself.” (P7) These participants used evidence pages more critically, treating them as a starting point for independent verification rather than confirmation. Others developed greater confidence in the agent through repeated interaction and prior trust in AI technology. “I have established a bit of confidence for the agent.” (P9) For these participants, evidence pages and highlighted information further reinforced their trust, sometimes leading them to accept the agent’s answer without fully checking whether all constraints were satisfied. “It was highlighted,

and the student option was already selected, so I just trusted it immediately.” (P11) In the future, HANSEL may need to actively prompt critical engagement with evidence to support accurate verification regardless of users’ trust orientations.

8 Discussion

We discuss the implications of our findings and the limitations of the current work.

8.1 Implications

Supporting critical engagement with evidence. After each task, we analyzed participants’ self-reported confidence. Participants reported higher confidence when using HANSEL compared to the baseline. While increased confidence is desirable when users correctly verify an answer, our findings suggest that confidence alone is not a reliable indicator of successful verification. In the wrong-answer condition, some participants still confidently submitted incorrect answers when using HANSEL, indicating a risk of miscalibrated trust [6, 43]. This points to a subtler issue than verification failure alone. The confidence participants developed when using HANSEL was not always grounded in independent confirmation of the agent’s reasoning; in some cases, it was grounded in the appearance of well-supported evidence. Titles, descriptions, and highlights signaled that the agent had a coherent rationale, and for some participants, that signal was sufficient. This aligns with the well-documented dynamic in AI-assisted decision-making that explanations for AI systems can sometimes contribute to overreliance [45]. Our qualitative findings reinforce this concern. Some participants treated evidence pages as a starting point for independent verification, while others used them to confirm an answer they were already inclined to accept. Even when participants opened the evidence pages, some did not detect the error. HANSEL succeeded in bringing users into the verification process, but engagement with evidence does not guarantee that users will identify what is wrong. Several design directions may help address this gap, such as flagging error-prone claim types or uncertain steps, or incorporating cognitive forcing functions that prompt users to inspect specific parts of the evidence before accepting an answer. More broadly, we see calibrating the relationship between evidence presentation and user judgment as an open question that future work on agent verification will need to address.

Continuity in human-agent collaboration. By exposing intermediate steps as interactive evidence artifacts, HANSEL enables more continuous user involvement throughout human-agent collaboration. Rather than treating the agent’s output as a fixed endpoint, HANSEL provides natural entry points for users to refine goals, adjust constraints, or explore alternatives. This allows users to intervene at the point where human effort is most efficient. For example, when a user wants to modify the location filter in an apartment search, rerunning the agent from scratch would require significant computation, whereas the user can directly adjust the relevant parameter on the evidence page with less effort. Beyond immediate intervention, HANSEL also reduces the need to rerun agents when users revisit previous tasks. Unlike screenshots, which capture a static snapshot that becomes outdated as web content

changes, HANSEL presents evidence as live, interactive pages reconstructed by replaying the agent’s actions on the current webpage. When users revisit a previous task, they can inspect the same evidence page to check whether conditions have changed, for example, whether a price has increased or a product has gone out of stock, without rerunning the agent. Together, these properties support a more cost-effective division of labor between humans and agents.

Beyond web tasks with bounded answers. While our evaluation focused on tasks with clear ground-truth answers in order to assess participants’ ability to detect and correct errors, we found many other interesting error types and verification needs from the agent trajectory analysis. One noteworthy aspect is that some agent answers cannot be verified through page evidence. For example, in the reasoning, AI agents sometimes mentioned “*Answering using domain knowledge*”, where the answer has no corresponding evidence pages to inspect. By surfacing the intermediate steps and key evidence pages from the agent’s trajectory, HANSEL makes this gap visible, allowing users to notice when supporting evidence is missing or incomplete. For tasks where correctness is subjective rather than verifiable against a ground truth, the notion of evidence should adapt to what is most useful for the user’s judgment. For example, in preference-driven tasks such as restaurant recommendation, HANSEL’s extraction pipeline could be adapted to identify decision steps as evidence, where users can assess whether the agent’s choices reflect their actual preferences and revise them if needed. More broadly, our findings suggest that evidence should be adapted to the structure of the task rather than tied to webpages themselves. We envision future agent systems presenting task-relevant evidence artifacts as interactive objects alongside agent outputs, enabling users to inspect, question, and revise.

8.2 Limitations

Our controlled user study has several limitations. While we made efforts to match task pairs by type and complexity, individual tasks may vary in difficulty. Learning effects are also threats to validity, given that participants could improve at verification and tool use over the course of the session. To mitigate this, all participants completed tasks in the same fixed order, and the alternating condition assignment ensured that any position-specific effects (e.g., fatigue) were distributed relatively evenly across both conditions. The participant sample is appropriate for an initial comparison of the two conditions, and future work could recruit a larger sample to examine smaller effects or subgroup differences.

The study setting also differs from everyday use of agent systems. Participants interacted with both interfaces through Zoom’s remote control feature, which may have introduced latency and reduced visual quality. The researcher was present throughout each session, and participants were explicitly asked to submit correct answers and were aware that their behavior was being observed. Although this setup likely increased verification effort compared with everyday use, it was necessary because in pilot studies where we did not emphasize reviewing the agent’s response, participants often bypassed the agent entirely and relied on external search, making it difficult to evaluate how the interface supports verification. We also did not allow participants to ask follow-up questions or use

other AI tools, both to preserve this evaluation focus and to keep tasks within the time limit.

HANSEL’s effectiveness depends on the quality of the agent’s trajectory and how well it aligns with users’ mental models of the task. When an agent follows an unexpected strategy, the evidence surfaced by HANSEL may leave users unsure where to begin verification. For example, in several pilot tasks involving restaurant or gym search, agents skipped review or mapping websites that participants expected to consult, leaving little useful evidence for verification within the interface. We therefore excluded such tasks from the formal study, but they remain important cases for future work and highlight the importance of training agents to adopt appropriate task-specific strategies. More broadly, our single-session lab study does not capture long-term adaptation, and HANSEL’s current BrowserView-based page reconstruction may not scale well to large-scale deployment.

9 Conclusion

We designed HANSEL, a system that presents evidence from web agent trajectories as interactive, verifiable artifacts. Rather than requiring users to passively read through overwhelming logs or trust LLM-generated summaries, HANSEL surfaces a minimal set of evidence pages that support the agent’s answer and presents them as live, navigable views with preserved page state. Our technical evaluation showed accurate evidence extraction ($F1 = 0.861$) with a 61.6% reduction in trajectory volume. A user study with 14 participants demonstrated that HANSEL significantly reduces completion time and effort while improving usability. We hope that HANSEL’s approach of surfacing interactive evidence for agent verification can inspire further research and adoption in real-world agent products, where robust verification support remains an open and pressing need.

References

- [1] Gagan Bansal, Jennifer Wortman Vaughan, Saleema Amershi, Eric Horvitz, Adam Fournery, Hussein Mozannar, Victor Dibia, and Daniel S. Weld. 2024. Challenges in Human-Agent Communication. *CoRR* abs/2412.10380 (2024). arXiv:2412.10380 doi:10.48550/ARXIV.2412.10380
- [2] Gagan Bansal, Tongshuang Wu, Joyce Zhou, Raymond Fok, Besmira Nushi, Ece Kamar, Marco Tulio Ribeiro, and Daniel Weld. 2021. Does the Whole Exceed its Parts? The Effect of AI Explanations on Complementary Team Performance. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems (CHI '21)*. Association for Computing Machinery, Article 81. doi:10.1145/3411764.3445717
- [3] Oliver Bentham, Nathan Stringham, and Ana Marasovic. 2024. Chain-of-Thought Unfaithfulness as Disguised Accuracy. *Transactions on Machine Learning Research* (2024). <https://openreview.net/forum?id=ydcP55u2e> Reproducibility Certification.
- [4] Samuel R. Bowman, Jeeyoon Hyun, Ethan Perez, Edwin Chen, Craig Pettit, Scott Heiner, Kamille Lukošiušė, Amanda Askill, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Christopher Olah, Daniela Amodei, Dario Amodei, Dawn Drain, Dustin Li, Eli Tran-Johnson, Jackson Kernion, Jamie Kerr, Jared Mueller, Jeffrey Ladish, Joshua Landau, Kamal Ndousse, Liane Lovitt, Nelson Elhage, Nicholas Schiefer, Nicholas Joseph, Noemi Mercado, Nova DasSarma, Robin Larson, Sam McCandlish, Sandipan Kundu, Scott Johnston, Shauna Kravec, Sheer El Showk, Stanislaw Fort, Timothy Telleen-Lawton, Tom Brown, Tom Henighan, Tristan Hume, Yuntao Bai, Zac Hatfield-Dodds, Ben Mann, and Jared Kaplan. 2022. Measuring Progress on Scalable Oversight for Large Language Models. arXiv:2211.03540 [cs.HC] <https://arxiv.org/abs/2211.03540>
- [5] browser-use. 2024. browser-use: Browser automation framework. <https://github.com/browser-use/browser-use>.
- [6] Zana Bućinca, Maja Barbara Malaya, and Krzysztof Z. Gajos. 2021. To Trust or to Think: Cognitive Forcing Functions Can Reduce Overreliance on AI in AI-assisted Decision-making. *Proc. ACM Hum.-Comput. Interact.* 5, CSCW1, Article 188 (April 2021). doi:10.1145/3449287
- [7] Alan Chan, Carson Ezell, Max Kaufmann, Kevin Wei, Lewis Hammond, Herbie Bradley, Emma Bluemke, Nitarshan Rajkumar, David Krueger, Noam Kolt, Lennart Heim, and Markus Anderljung. 2024. Visibility into AI Agents. arXiv:2401.13138 [cs.CY] <https://arxiv.org/abs/2401.13138>
- [8] Andy Chung, Yichi Zhang, Kaixiang Lin, Aditya Rawal, Qiaozi Gao, and Joyce Chai. 2025. Evaluating Long-Context Reasoning in LLM-Based WebAgents. arXiv:2512.04307 [cs.LG] <https://arxiv.org/abs/2512.04307>
- [9] Victoria Clarke and Virginia Braun. 2017. Thematic analysis. *The Journal of Positive Psychology* 12, 3 (2017), 297–298.
- [10] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2Web: Towards a Generalist Agent for the Web. arXiv:2306.06070 [cs.CL] <https://arxiv.org/abs/2306.06070>
- [11] Electron Team. 2025. Electron. <https://www.electronjs.org/>.
- [12] Will Epperson, Gagan Bansal, Victor C Dibia, Adam Fournery, Jack Gerrits, Erkan Zhu, and Saleema Amershi. 2025. Interactive Debugging and Steering of Multi-Agent AI Systems. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. ACM, 1–15. doi:10.1145/3706598.3713581
- [13] Martin Eppler and Jeanne Mengis. 2004. The Concept of Information Overload: A Review of Literature From Organization Science, Accounting, Marketing, MIS, and Related Disciplines. *Inf. Soc.* 20 (11 2004), 325–344. doi:10.1080/01972240490507974
- [14] K. J. Kevin Feng, Kevin Pu, Matt Latzke, Tal August, Pao Siangliulue, Jonathan Bragg, Daniel S. Weld, Amy X. Zhang, and Joseph Chee Chang. 2025. Cocoa: Co-Planning and Co-Execution with AI Agents. arXiv:2412.10999 [cs.HC] <https://arxiv.org/abs/2412.10999>
- [15] Krzysztof Z. Gajos and Lena Mamykina. 2022. Do People Engage Cognitively with AI? Impact of AI Assistance on Incidental Learning. In *27th International Conference on Intelligent User Interfaces (IUI '22)*. ACM, 794–806. doi:10.1145/3490099.3511138
- [16] Google. 2025. Gemini Deep Research. <https://gemini.google/overview/deep-research/>.
- [17] Google. 2025. Get started with Gemini. <https://support.google.com/gemini/answer/14143489>.
- [18] Izzeddin Gur, Ulrich Rueckert, Aleksandra Faust, and Dilek Hakkani-Tur. 2018. Learning to Navigate the Web. arXiv:1812.09195 [cs.LG] <https://arxiv.org/abs/1812.09195>
- [19] Gaole He, Gianluca Demartini, and Ujwal Gadiraju. 2025. Plan-Then-Execute: An Empirical Study of User Trust and Team Performance When Using LLM Agents As A Daily Assistant. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. ACM, 1–22. doi:10.1145/3706598.3713218
- [20] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. 2024. WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models. arXiv:2401.13919 [cs.CL] <https://arxiv.org/abs/2401.13919>
- [21] Faria Huq, Zora Zhiruo Wang, Frank F. Xu, Tianyue Ou, Shuyan Zhou, Jeffrey P. Bigham, and Graham Neubig. 2025. CowPilot: A Framework for Autonomous and Human-Agent Collaborative Web Navigation. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (System Demonstrations)*. Association for Computational Linguistics, 163–172. doi:10.18653/v1/2025.naacl-demo.17
- [22] Sayash Kapoor, Benedikt Stroebel, Peter Kirgis, Nitya Nadgir, Zachary S Siegel, Boyi Wei, Tianci Xue, Ziru Chen, Felix Chen, Saiteja Utpala, Franck Ndongma, Dheeraj Oruganty, Sophie Lusk, Kangheng Liu, Botao Yu, Yong Dai, Dongyoon Hahm, Harsh Trivedi, Huan Sun, Juyong Lee, Tengjun Jin, Yifan Mai, Yifei Zhou, Yuxuan Zhu, Rishi Bommasani, Daniel Kang, Dawn Song, Peter Henderson, Yu Su, Percy Liang, and Arvind Narayanan. 2025. Holistic Agent Leaderboard: The Missing Infrastructure for AI Agent Evaluation. arXiv:2510.11977 [cs.AI] <https://arxiv.org/abs/2510.11977>
- [23] Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, Danny Hernandez, Dustin Li, Esin Durmus, Evan Hubinger, Jackson Kernion, Kamille Lukošiušė, Karina Nguyen, Newton Cheng, Nicholas Joseph, Nicholas Schiefer, Oliver Rausch, Robin Larson, Sam McCandlish, Sandipan Kundu, Saurav Kadavath, Shannon Yang, Thomas Henighan, Timothy Maxwell, Timothy Telleen-Lawton, Tristan Hume, Zac Hatfield-Dodds, Jared Kaplan, Jan Brauner, Samuel R. Bowman, and Ethan Perez. 2023. Measuring Faithfulness in Chain-of-Thought Reasoning. arXiv:2307.13702 [cs.AI] <https://arxiv.org/abs/2307.13702>
- [24] Brian Y. Lim, Anind K. Dey, and Daniel Avrahami. 2009. Why and why not explanations improve the intelligibility of context-aware intelligent systems. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '09)*. Association for Computing Machinery, 2119–2128. doi:10.1145/1518701.1519023
- [25] Xing Han Lü, Amirhossein Kazemnejad, Nicholas Meade, Arkil Patel, Dongchan Shin, Alejandra Zambrano, Karolina Stanczak, Peter Shaw, Christopher Pal, and Siva Reddy. 2025. AgentRewardBench: Evaluating Automatic Evaluations of Web Agent Trajectories. arXiv:2504.08942 [cs.CL] <https://arxiv.org/abs/2504.08942>
- [26] Brian Lubars and Chenhao Tan. 2019. Ask Not What AI Can Do, But What AI Should Do: Towards a Framework of Task Delegability. arXiv:1902.03245 [cs.AI]

- <https://arxiv.org/abs/1902.03245>
- [27] Andreas Madsen, Sarath Chandar, and Siva Reddy. 2024. Are self-explanations from Large Language Models faithful? In *Findings of the Association for Computational Linguistics, ACL 2024, Bangkok, Thailand and virtual meeting, August 11-16, 2024 (Findings of ACL)*. Association for Computational Linguistics, 295–337. doi:10.18653/v1/2024.FINDINGS-ACL.19
 - [28] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2023. GAIA: a benchmark for General AI Assistants. arXiv:2311.12983 [cs.CL] <https://arxiv.org/abs/2311.12983>
 - [29] Hussein Mozannar, Gagan Bansal, Cheng Tan, Adam Fourney, Victor Dibia, Jingya Chen, Jack Gerrits, Tyler Payne, Matheus Kunzler Maldaner, Madeleine Grunde-McLaughlin, Eric Zhu, Griffin Bassman, Jacob Alber, Peter Chang, Ricky Loynd, Friederike Niedtner, Ece Kamar, Maya Murad, Rafah Hosn, and Saleema Amershi. 2025. Magentic-UI: Towards Human-in-the-loop Agentic Systems. arXiv:2507.22358 [cs.AI] <https://arxiv.org/abs/2507.22358>
 - [30] Shikhar Murty, Hao Zhu, Dzhmitry Bahdanau, and Christopher D. Manning. 2025. NNetNav: Unsupervised Learning of Browser Agents Through Environment Interaction in the Wild. arXiv:2410.02907 [cs.CL] <https://arxiv.org/abs/2410.02907>
 - [31] Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, Xu Jiang, Karl Cobbe, Tyna Eloundou, Gretchen Krueger, Kevin Button, Matthew Knight, Benjamin Chess, and John Schulman. 2022. WebGPT: Browser-assisted question-answering with human feedback. arXiv:2112.09332 [cs.CL] <https://arxiv.org/abs/2112.09332>
 - [32] Liangbo Ning, Ziran Liang, Zhuohang Jiang, Haohao Qu, Yujuan Ding, Wenqi Fan, Xiaoyong Wei, Shanru Lin, Hui Liu, Philip S. Yu, and Qing Li. 2025. A Survey of WebAgents: Towards Next-Generation AI Agents for Web Automation with Large Foundation Models. arXiv:2503.23350 [cs.AI] <https://arxiv.org/abs/2503.23350>
 - [33] OpenAI. 2025. Introducing Operator. <https://openai.com/index/introducing-operator/>.
 - [34] OpenHands. 2025. Trajectory Visualizer. <https://github.com/OpenHands/trajectory-visualizer>.
 - [35] Tianyue Ou, Wanyao Guo, Apurva Gandhi, Graham Neubig, and Xiang Yue. 2025. AgentDiagnose: An Open Toolkit for Diagnosing LLM Agent Trajectories. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, 207–215. doi:10.18653/v1/2025.emnlp-demos.15
 - [36] Marvin Paffa, Kate Larson, and Mark Hancock. 2024. Unraveling the Dilemma of AI Errors: Exploring the Effectiveness of Human and Machine Explanations for Large Language Models. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems (CHI '24)*. Association for Computing Machinery, Article 839. doi:10.1145/3613904.3642934
 - [37] Yi-Hao Peng, Dingzeyu Li, Jeffrey P Bigham, and Amy Pavel. 2025. Morae: Proactively Pausing UI Agents for User Choices. In *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology (UIST '25)*. Association for Computing Machinery, New York, NY, USA, Article 198, 14 pages. doi:10.1145/3746059.3747797
 - [38] Peter Pirolli and Stuart Card. 2005. The sensemaking process and leverage points for analyst technology as identified through cognitive task analysis. In *Proceedings of international conference on intelligence analysis*. McLean, VA, USA.
 - [39] Tianlin Shi, Andrej Karpathy, Linxi Fan, Jonathan Hernandez, and Percy Liang. 2017. World of Bits: An Open-Domain Platform for Web-Based Agents. In *Proceedings of the 34th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 70)*. PMLR, 3135–3144. <https://proceedings.mlr.press/v70/shi17a.html>
 - [40] Chenglei Si, Navita Goyal, Tongshuang Wu, Chen Zhao, Shi Feng, Hal Daumé Iii, and Jordan Boyd-Graber. 2024. Large Language Models Help Humans Verify Truthfulness – Except When They Are Convincingly Wrong. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Association for Computational Linguistics, 1459–1474. doi:10.18653/v1/2024.naacl-long.81
 - [41] John Sweller. 1988. Cognitive Load During Problem Solving: Effects on Learning. *Cognitive Science* 12, 2 (1988), 257–285.
 - [42] Sree Harsha Tanneru, Dan Ley, Chirag Agarwal, and Himabindu Lakkaraju. 2024. On the Hardness of Faithful Chain-of-Thought Reasoning in Large Language Models. arXiv:2406.10625 [cs.CL] <https://arxiv.org/abs/2406.10625>
 - [43] Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. 2023. Language models don’t always say what they think: unfaithful explanations in chain-of-thought prompting. In *Proceedings of the 37th International Conference on Neural Information Processing Systems (NIPS '23)*. Curran Associates Inc., Article 3275.
 - [44] Karthik Valmeekam, Sarath Sreedharan, Matthew Marquez, Alberto Olmo, and Subbarao Kambhampati. 2023. On the Planning Abilities of Large Language Models (A Critical Investigation with a Proposed Benchmark). arXiv:2302.06706 [cs.AI] <https://arxiv.org/abs/2302.06706>
 - [45] Helena Vasconcelos, Matthew Jörke, Madeleine Grunde-McLaughlin, Tobias Gerstenberg, Michael Bernstein, and Ranjay Krishna. 2023. Explanations Can Reduce Overreliance on AI Systems During Decision-Making. arXiv:2212.06823 [cs.HC] <https://arxiv.org/abs/2212.06823>
 - [46] Xinru Wang, Ming Yin, Eunye Koh, and Mustafa Doga Dogan. 2026. XAgent: An Explainability Tool for Identifying and Correcting Failures in Multi-Agent Workflows. arXiv:2512.17896 [cs.HC] <https://arxiv.org/abs/2512.17896>
 - [47] Tongshuang Wu, Michael Terry, and Carrie J. Cai. 2022. AI Chains: Transparent and Controllable Human-AI Interaction by Chaining Large Language Model Prompts. arXiv:2110.01691 [cs.HC] <https://arxiv.org/abs/2110.01691>
 - [48] Yiheng Xu, Dunjie Lu, Zhennan Shen, Junli Wang, Zekun Wang, Yuchen Mao, Caiming Xiong, and Tao Yu. 2025. AgentTrek: Agent Trajectory Synthesis via Guiding Replay with Web Tutorials. arXiv:2412.09605 [cs.CL] <https://arxiv.org/abs/2412.09605>
 - [49] Tianci Xue, Weijian Qi, Tianneng Shi, Chan Hee Song, Boyu Gou, Dawn Song, Huan Sun, and Yu Su. 2025. An Illusion of Progress? Assessing the Current State of Web Agents. arXiv:2504.01382 [cs.AI] <https://arxiv.org/abs/2504.01382>
 - [50] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2023. WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents. arXiv:2207.01206 [cs.CL] <https://arxiv.org/abs/2207.01206>
 - [51] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net. https://openreview.net/forum?id=WE_vluYUL-X
 - [52] Ori Yoran, Samuel Joseph Amouyal, Chaitanya Malaviya, Ben Bogin, Ofir Press, and Jonathan Berant. 2024. AssistantBench: Can Web Agents Solve Realistic and Time-Consuming Tasks? arXiv:2407.15711 [cs.CL] <https://arxiv.org/abs/2407.15711>
 - [53] Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. 2024. GPT-4V(ision) is a Generalist Web Agent, if Grounded. arXiv:2401.01614 [cs.IR] <https://arxiv.org/abs/2401.01614>
 - [54] Runtao Zhou, Giang Nguyen, Nikita Kharya, Anh Totti Nguyen, and Chirag Agarwal. 2026. Improving Human Verification of LLM Reasoning through Interactive Explanation Interfaces. arXiv:2510.22922 [cs.HC] <https://arxiv.org/abs/2510.22922>
 - [55] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. arXiv:2307.13854 [cs.AI] <https://arxiv.org/abs/2307.13854>